

Algorithm Analysis

Learning Outcomes

- Explain the basic techniques for the analysis of algorithms;
- Analyze different algorithms in terms of the amount of resource they consume in order to solve problems
- Determine when and how to use various data structures including linked lists, queues, stacks, binary trees, and binary search trees
- Use different simple and advanced sorting and searching algorithms
- Apply the right combination of data structures and algorithms for different information processing tasks.

Definition (Algorithm)

- What exactly is an algorithm?
 - An algorithm is a clearly specified set of simple instructions to be followed to solve a problem
 - Any well-defined *computational procedure* that *takes some value (or set of values* called input) as an input and *produces some value (or set of values* called output) as an output
 - A sequence of *computational steps* that *transforms the input* into the *output*
 - A set of rules (well-defined, finite) used for calculation or problem solving (for example with computer)

Definition (Algorithm)

- What exactly is an algorithm?
 - A finite set of instructions that, if followed, accomplishes a particular task
 - Is a precise, systematic method for producing a specified result
 - Is a tool for solving a well-defined computational problem
 - The statement of the problem specifies (in general terms) the desired input/output relationships
 - The algorithm describes a specific computational procedure for achieving the input/output relationships

Example Algorithm (1)

- **The sorting problem**

- Statement of the sorting problem

- Input: a sequence of n number $\langle a_1, a_2, \dots, a_n \rangle$
- Output: a permutation (reordering) $\langle a_1', a_2', \dots, a_n' \rangle$ such that $a_1' \leq a_2' \leq \dots \leq a_n'$.

- That is, given an input sequence such as $\langle 31, 41, 59, 26, 41, 58 \rangle$ a sorting algorithm returns as an output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$

Example Algorithm (1)

- **A sorting algorithm (insertion sort)**

INSERTION-SORT(A)

```
1.  for  $j = 2$  to  $\text{length}[A]$ 
2.      do  $key \leftarrow A[j]$ 
3.          //insert  $A[j]$  to sorted sequence  $A[1..j-1]$ 
4.           $i \leftarrow j-1$ 
5.          while  $i > 0$  and  $A[i] > key$ 
6.              do  $A[i+1] \leftarrow A[i]$  //move  $A[i]$  one position
           right
7.               $i \leftarrow i-1$ 
8.           $A[i+1] \leftarrow key$ 
```

Example Algorithm (2)

- **Finding the maximum element problem**
 - Input: an array A storing n integers
 - Output: the maximum element in A
 - That is, given array $A=[31, 41, 26, 41, 58]$, the maximum algorithm returns 58 which is the maximum element in the array

Example Algorithm (2)

An algorithm for finding the maximum element

Input: An array A storing n integers

Output: The maximum element in A

find_max(A,n):

currentMax A[0] ←

for i=1 to n-1 Do:

if A[i] > currentMax Then

currentMax=A[i]

return currentMax

Example Algorithm (3)

- **Finding the greatest common divisor problem of two natural numbers**
 - Problem statement
 - Determine the greatest common divisor of two natural numbers x and y (where $x < y$)
 - Input
 - Two natural numbers a and b
 - Output
 - The largest natural number d which is the common divisor of a and b

Example Algorithm (3)

- **Algorithm for finding the greatest common divisor of two natural numbers problem**
 1. Divide y by x with remainder r
 2. Replace y by x , and x by r
 3. Repeat step 1 until r is zero
- **When this algorithm terminates, y is the highest common factor of**

Example Algorithm (3)

GCD

```
int gcd(int n, int m){  
    // n>0 and m>0  
  
    while (m != n)  
        if (n>m)  
            n = n - m;  
        else  
            m = m - n;  
  
    return n;  
}
```

- $\text{GCD}(33,21)=3$
- $33 = 1*21 + 12$
- $21 = 1*12 + 9$
- $12 = 1*9 + 3$
- $9 = 3*3$

Purpose Algorithm in programs

- To accept input
 - There are zero or more quantities which are externally supplied
- To change values hold by data structures
 - It is through an algorithm that you change values
- To produce an output
 - At least one quantity is produced
- To reorganize or reshuffle the data structure
 - Sorting
 - Deleting an item from the data structure
 - Adding an item into the data structure

Properties of Algorithms

- Every algorithm possesses the following properties
 - I/O
 - Finiteness
 - Definiteness
 - Effectiveness
 - Correctness
 - Efficiency
 - Sequence (sequential)
 - simplicity
 - Language Independence
 - Completeness

Essential Properties of Algorithms

- **Input specified**

- The inputs are the data that will be transformed during the computation to produce the output
- Every algorithm should have a specified number (zero or more) input values (or quantities) which are externally supplied
- We must specify
 - the type of data
 - The amount of data

Essential Properties of Algorithms

- **Output specified**

- The outputs are the data resulting from the computation (the intended result)
- Every algorithm should have one or more output procedures
- A possible output for some computations is a statement that there can be no output, i.e., no solution is possible, i.e., at least one quantity is produced

Essential Properties of Algorithms

- **Finiteness (It Must Terminate)**
 - An algorithm must have the property of finiteness
 - That is, every valid algorithm should have finite number of steps
 - It must complete after a finite number of steps
 - It must eventually stop either with the right output or with a statement that no solution is possible
 - If you trace out the instructions of an algorithm, then for all cases the algorithm must terminate after a finite number of steps
 - Finiteness is an issue for computer algorithms because if the algorithm doesn't specify when to stop (computer algorithms often repeat instructions), the computer will continue to repeat the instructions for ever

Essential Properties of Algorithms

- **Definiteness (Absence of Ambiguity)**
 - Definiteness means specifying the sequence of operations for transforming the inputs into the outputs
 - Algorithms must specify every step
 - Every step in the algorithm has to be clearly defined
 - Each step must be clearly defined, having **one and only one interpretation**
 - Every detail of each step must be spelled out, including how to handle errors

Essential Properties of Algorithms

- **Definiteness (Absence of Ambiguity)**
 - At each point in computation, one should be able to tell exactly what happens next
 - **It must be composed of concrete steps**
 - There can be no ambiguity as to which step will be performed
 - **Definiteness ensures that if the algorithm is performed at different times or by different agents (people or computers) using the same data, the output will be the same**

Essential Properties of Algorithms

- **Effectiveness (Feasibility)**
 - It must be possible to perform each step exactly and in a finite amount of time.
 - It is not enough that each operation be definite as in definiteness, but it must also be feasible
 - It must be possible to perform each instruction
 - Whereas definiteness specifies which operations to do, effectiveness means that they are doable

Essential Properties of Algorithms

- **Correctness**

- An algorithm must compute correct answer for all possible legal inputs
- Is the most important criteria for evaluating algorithms
- If the algorithm is not doing what it is supposed to do, it is worthless

Essential Properties of Algorithms

- **Efficiency (focus of this chapter)**
 - An algorithms should make efficient use of the computer resources
 - In particular, we would like an algorithm to run as fast as possible using as little memory as possible
 - An algorithm must solve a problem with the least amount of computational resources such as time and space.
 - To keep things simple, we will concentrate on the running time of algorithms and will not look at the space (the amount of memory needed)

Essential Properties of Algorithms

- **Sequential**

- Every algorithm should have a beginning (start step) and a halt (end) step
- Between the two every step should have preceding and succeeding steps
- That is, Each step must have a unique defined preceding and succeeding step
- The first step (start step) and last step (halt step) must be clearly noted

Essential Properties of Algorithms

- **Simplicity**

- We would like our algorithms to be easy to understand and easy to implement
- There often (but not always) a trade off between efficiency and simplicity
- More efficient algorithms tend to be more complicated
- It depends on particular task which of the two criteria is more important

Essential Properties of Algorithms

- **Language Independence**
 - It must not depend on any one programming language
- **Completeness**
 - must solve the problem completely
- Any process having these properties will be called an algorithm

Languages in algorithms

- Because algorithms are developed by people, but executed by some other agent, they must be written in some language
- The most important requirement of the language is that the person who creates the instructions and the agent that performs them interpret the instructions the same way
- An algorithm can be described in many ways
 - Natural language
 - Programming language
 - Pseudocode

Analysis of algorithms

- What does it mean by analyzing algorithms?
 - It means predicting the resources that the algorithm requires
- **Occasionally**, resources such as **memory**, **communication**, bandwidth etc are primary concern in analyzing resources that an algorithm requires
- **Most often**, computational time is the primary concern in analyzing resources

Analysis of algorithms

- The objective of algorithm analysis is
 - to determine how quickly an algorithm executes in practice
 - To measure either time or space requirements of an algorithm
- What to measure
 - Space utilization- amount of memory required
 - Time efficiency- amount of time required to process the data

Analysis of algorithms

- What to analyze
 - The most important recourse to analyze is generally the running time
 - Several factors affect the running time of an a program
 - Compiler used
 - Computer used
 - The algorithm used
 - The input to the algorithm
 - The first two are beyond the scope of theoretical model
 - Although thy are important, we will not deal with them in the course
 - The last two are the main factors that we deal
 - Typically, the size of the input is an important consideration

Analysis of algorithms

- Space utilization and Time efficiency depends on many factors
 - Size of input
 - Speed of machine (computer used)
 - Quality of source code (used algorithm)
 - Quality of compiler
- Focuses on measuring the running time of an algorithm

Running Time of Algorithm

- The running time of an algorithm depends on a number of factors
- The running time of an algorithm is affected by the factors mentioned in the previous slide
- But, for most algorithm, the running time depends on the input
 - An already sorted sequence is easier to sort
- for most algorithm, the running time depends on size of the input
 - Short sequences are easier to sort than long ones
 - That is the running time of most algorithms varies with the input and typically grows with the input size

Running Time of Algorithm

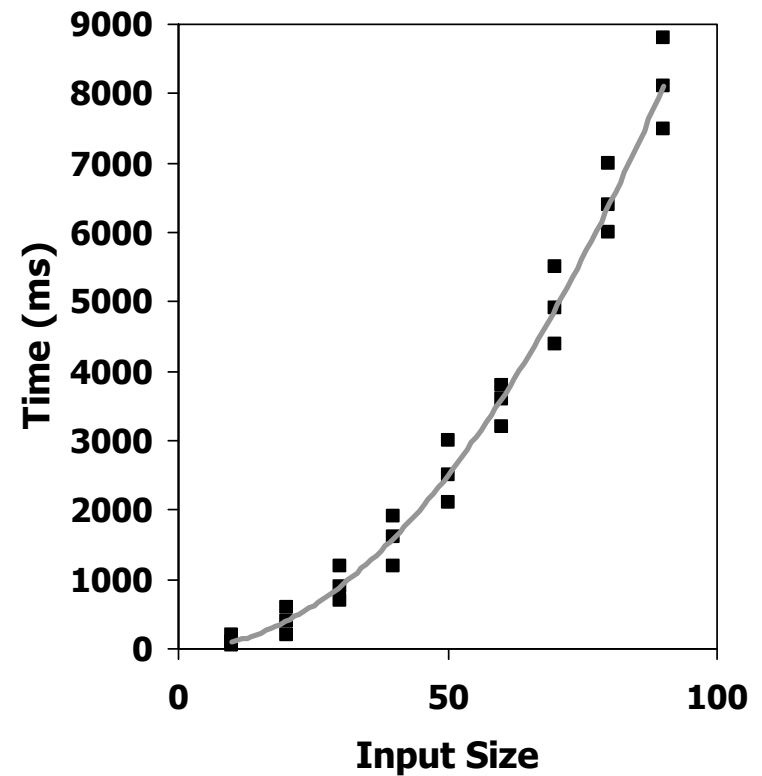
- Generally, we seek an upper bound on the running time, because everybody likes a guarantee
- We can measure the running time of an algorithm in two ways
 - Empirically
 - Theoretically

Empirical Analysis

- What you should do
 - Write a program that implements the algorithm
 - Run the program with the data sets (the inputs) of varying size and composition
 - Use the method like `clock()` (or `System.currentTimeMillis()`) to get an accurate measure of the running time
 - Plot the results

Empirical Analysis

- The resulting data set should look something like this



Empirical Analysis

- Is an experimental study
- The result comes from the running time of the program
- It uses system time

Limitations of Empirical Analysis

- It can't be used in estimating the efficiency of algorithms because the **result varies due to variations in**
 - Processor speed
 - Input size
 - Current processor load
 - SW environment
- So, in order to compare two algorithms, the same hardware and software environment must be used

Limitations of Empirical Analysis

- Involves implementing the algorithm and testing it on various instances
 - It is necessary to implement and test the algorithm in order to determine its running time
- But, implementing the algorithm may be difficult
- This is not the case for theoretical analysis

Limitations of Empirical Analysis

- The difficulty to know which instances to test it on
- Experiments can be done only on a limited set of inputs, and may not be indicative of the running time on other inputs not included in the experiment
- That is, results may not be indicative of the running time on other inputs not included in the experiment

Theoretical Analysis

- Is in contrast to the “experimental approach”
- Allows us to evaluate the speed (efficiency) of an algorithm in a way that is independent from the HW and SW environment
- Is a general methodology for analyzing the running time of an algorithm
- Uses a high-level description of the algorithm instead of testing one of its implementation
- Takes into account all possible inputs

Theoretical Analysis

- Takes an algorithm and produces a function $T(n)$ which depends on the number of operations
- That is, running time is expressed as $T(n)$ for some function T on input size n
- The output comes from the quantity of resources using mathematical concepts
- Does not use system time

Theoretical Analysis

- Rather it uses the number of operations (which are expressed in time units) because the number of operations do not vary with
 - Processor speed
 - Input size
 - Current processor load
 - SW environment
- Theoretical analysis allows us to separate the effect of these factors

Theoretical Analysis

- What will be the number of operations for the algorithm below using a computer having
 - 100 MHZ and 750 MHZ
 - Using DOS OS
 - Using UNIX OS
 - While printing, browsing

```
for(i=1; i <= 100; i++)  
    cout << i;
```

Theoretical Analysis

- Is an approach that can be used to measure or estimate the complexity of an algorithm as the approach do not vary due to variations in the computer systems
- The only factor that affects measure of complexity is **the input size of the algorithm**
- To avoid this, measure complexity of algorithm for arbitrary number n as $n \rightarrow \infty$

How do measure complexity of algorithms

- Two steps or phases for this
 - Analysis of algorithm (i.e., theoretical analysis)
 - Order of magnitude

How do measure complexity of algorithms

- Step one - analysis of algorithm
 - At this stage we should have an algorithm or select an algorithm
 - By analyzing this algorithm we end up with a function $T(n)$, which is the computing time of an algorithm for input size n
 - $T(n)$ = the number of operations of time units
 - This first step is used to measure the complexity of the algorithm

Calculating complexity of an algorithm

- Step one - analysis of algorithm
 - In analyzing an algorithm written in pseudocode to determine its complexity, determine the number of operations on the underlying model of computation
 - To obtain an estimate of this number, we **count primitive operations**, chosen to reflect the complexity of the total cost of all operations

Calculating complexity of an algorithm

Step one - analysis of algorithm

– **Primitive operations**

- Basic computations performed by an algorithm
- Identifiable in pseudocode
- Largely independent from the programming language
- Exact definition not important

Primitive or Basic Operations

- Step one - analysis of algorithm
 - **Examples of basic operations could be**
 - **An assignment** (Assigning a value to a variable)
 - **Calling a method**
 - **Returning from a method**
 - **Evaluating an expression**
 - An **arithmetic operation** between two variable (e.g., Performing an arithmetic operations such as addition)
 - **A Comparison between to variables (two numbers)**
 - **Indexing into an array**

Primitive or Basic Operations

- By inspecting the pseudo-code, we can **count** the number of primitive operations executed by an algorithm.
- Example:

Algorithm arrayMax(A, n):

Input: An array A storing n integers.

Output: The maximum element in A .

$currentMax \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $currentMax < A[i]$ **then**

$currentMax \leftarrow A[i]$

return $currentMax$

⋈

Running time- $T(n)$

- We measure execution (running) time in terms of any of the following
 - Arithmetic operations
 - Assignment operations
 - Loop iterations
 - Comparisons
 - Procedure calls (calling a method)
 - Returning from a method

Order of Magnitude

- Refers to the rate at which the storage or time grows as a function of problem size
- Input: $T(n)$, the timing function
- Output: order of $T(n)$ written as $O(T(n))$
- **The output of helps to determine the category to which the algorithm belongs to some known functions (functions whose complexity is known through research and experiment)**
- Thus, the output is expressed in terms of its relationship to some known functions

Order of Magnitude

- This type of analysis is called asymptotic analysis
- For large enough inputs, the multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself
- When we look at input sizes large enough to make only the order of growth of the running time relevant, we are studying asymptotic efficiency of algorithms
- That is, we are concerned with how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound
- Usually, an algorithm that is asymptotically more efficient will be the best choice for all but very small inputs

More examples on computing $T(n)$ and $O(T(n))$

- If we had to perform all these work every time we need to analyze a program, the task would quickly be infeasible
- Since, we are giving the answer in terms of Big-Oh, there are lots of short cuts that can be taken without affecting the final answer
- For example- consider example...
 - Line 6 is obviously $O(1)$ statement (per execution), so it is silly to count precisely whether it is two, three or four units- it doesn't matter
 - Line 3 is obviously insignificant compared to the four loop, so it is silly to waste time there
- **This leads to several obvious general rules also called analysis rules**

Analysis Rules

- These are guidelines for finding out the time complexity of a piece of code
1. **We assume an arbitrary time unit**
 2. **Unlike real computers, it takes exactly one time unit to do any thing (simple).** Thus, execution of one of the following operations takes time one (1)
 - assignment statements
 - Single input/output statements
 - Single Boolean expression evaluation
 - Single arithmetic
 - function returns

Analysis Rules

- Weakness of the above assumption
 - In real life, not all operations take exactly the same time
 - In our model (i.e., according to the assumption), one disk read counts the same as an addition, even though the addition is typically several orders of magnitude faster

Analysis Rules

3. If-then-else statement

- The running time of an if-then-else statement is never more than the running time of the test plus the larger of the running times of S1 and S2
 - The **total time for condition evaluation** + the **greater complexity** of then/else clause
 - The total time for condition evaluation + the maximum of the running times for the individual clauses in the selection
 - The total time for condition evaluation + max (time taken by alternatives)
- Clearly, this can be an over-estimate in some cases, but it is never an under estimate

Analysis Rules

- **Example - Running time of if-then- else statement**

executed n times {

If ($x > 5$)

for ($i = 1; i \leq n; i++$)

$\text{cout} \ll i$ — constant time

else

$\text{cout} \ll \text{"hello"};$ — constant time

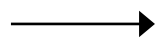
– Total time = $1 + \max(n, 1) = 1 + n = O(n)$

Analysis Rules

- **Example - Running time of if-then- else statement**

Worst-case running time: the test, plus *either* the **then** part *or* the **else** part (whichever is the larger).

test:
constant



```
if (depth( ) != otherStack.depth( ) ) {  
    return false;  
}  
else {  
    for (int n = 0; n < depth( ); n++) {  
        if (!list[n].equals(otherStack.list[n]))  
            return false;  
    }  
}
```



then part:
constant



else part:
(constant +
constant) * n

another if :
constant +
constant
(no else part)

$$\text{Total time} = c_0 + c_1 + (c_2 + c_3) * n = \mathbf{O(n)}$$

Analysis Rules

4. Consecutive statements

Add the time complexities of each statement.

- These just add (which means that the maximum is the one that counts)

constant time	→	<code>x = x + 1;</code>		
		<code>for (i=1; i<=n; i++) {</code>	}	executed <i>n</i> times
constant time	→	<code> m = m + 2;</code>		
		<code> }</code>		
outer loop executed <i>n</i> times	{	<code> for (i=1; i<=n; i++) {</code>	}	inner loop executed <i>n</i> times
		<code> for (j=1; j<=n; j++) {</code>		
		<code> k = k + 1;</code>		
		<code> }</code>		
		<code> }</code>		
		<code>}</code>		

constant time

$$\text{Total time} = c_0 + c_1n + c_2n^2 = \mathbf{O(n^2)}$$

Analysis Rules

- **Example - Consecutive statements**

```
for (int i=0; i<n; i++)  
    A[i]=0;  
for (i=0; i<n; i++)  
{  
    for (int j=0; j<n; j++)  
    {  
        A[i] += i + j;  
    }  
}
```

- This program fragment, which has $O(n)$ work followed by $O(n^2)$ work, is also $O(n^2)$

Analysis Rules

5. Running time of a Loop statement

The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

executed n times {
 for ($i=1; i \leq n; i++$)
 {
 $m = m + 2;$ ← constant time
 }

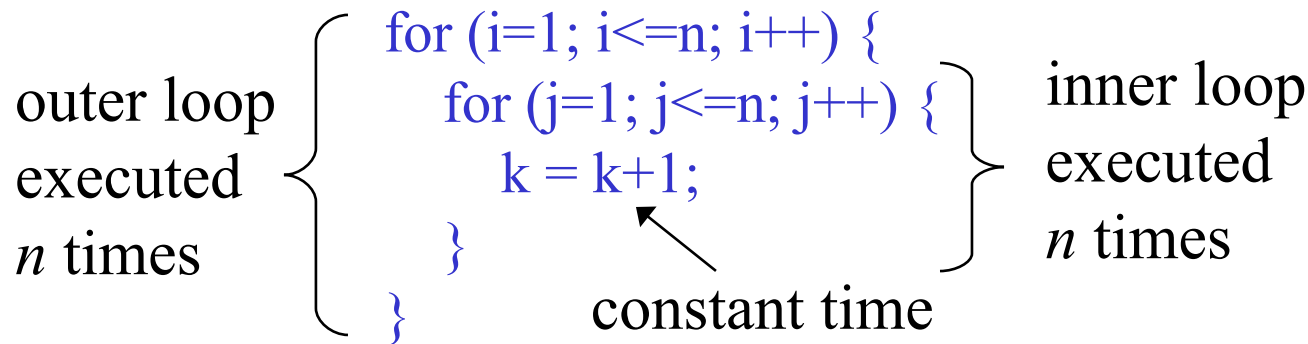
Total time = a constant $c * n = cn = \mathbf{O(n)}$

- Always assume the loop executes the maximum number of times

Analysis Rules

6. Running time of Nested Loop

- Analyze inside out
- The running time of a statement inside a group of nested loops is the running time of the statement multiplied by the product of the sizes of all the loops



$$\text{Total time} = c * n * n * = cn^2 = \mathbf{O(n^2)}$$

Total running time is the product of the sizes of all the loops.

Analysis Rules

7. Function call

- Running time of a function call is
 - $1 + \text{time taken for parameter calculation} + \text{time required for the execution of function body (body time)}$

8. while loop – analyze like for loop

9. Switch statement- take the complexity of the most expensive case

Analysis Rules

- Example
 - Find the complexity of the algorithm below using the analysis rules discussed earlier
 - Find the actual complexity for this algorithm and compare it with the result obtained in (a)

```

int count(){
    int k=0;
    cout<< "Enter an
integer";
    cin>>n;
    for (i=0;i<n;i++)
        k=k+1;
    return 0;
}

```

Time Units to Compute

1 for the assignment statement:

int k=0

1 for the output statement.

1 for the input statement.

In the for loop:

1 assignment, $n+1$ tests, and n increments.

n loops of 2 units for an assignment, and an addition.

1 for the return statement.

$$T(n) = 1 + 1 + 1$$

3. void func()

```
{  
  int x=0;  
  int i=0;  
  int j=1;  
  cout<< "Enter an Integer value";  
  cin>>n;  
  while (i<n){  
    x++;  
    i++;  
  }  
  while (j<n)  
  {  
    j++;  
  }  
}
```

Time Units to Compute

-

1 for the first assignment statement:

x=0;

1 for the second assignment statement: i=0;

1 for the third assignment statement: j=1;

1 for the output statement.

1 for the input statement.

In the first while loop:

n+1 tests

n loops of 2 units for the two increment (addition) operations

In the second while loop:

n tests

n-1 increments

$$T(n) = 1 + 1 + 1 + 1 + 1 + n + 1 + 2n + n + n - 1 = 5n + 5 = O(n)$$

2

```
int total(int n)  
{  
  int sum=0;  
  for (int i=1;i<=n;i++)  
    sum=sum+1;  
  return sum;  
}
```

Time Units to Compute

-

1 for the assignment statement: int sum=0

In the for loop:

1 assignment, $n+1$ tests, and n increments.

n loops of 2 units for an assignment, and an addition.

1 for the return statement.

$T(n) = 1 + (1 + n + 1 + n) + 2n + 1 = 4n + 4 = O(n)$

14. int sum (int n)

```
{  
  int partial_sum = 0;  
  for (int i = 1; i <= n; i++)  
    partial_sum = partial_sum  
+ (i * i * i);  
  return partial_sum;  
}
```

$$+(1+n+1+n)+2n+1 = 4n+6 = O(n)$$

Time Units to Compute

-

1 for the assignment.

1 assignment, $n+1$ tests, and n increments.

n loops of 4 units for an assignment, an addition, and two multiplications.

1 for the return statement.

$$T(n) = 1 + (1+n+1+n) + 4n + 1 = 6n + 4 = O(n)$$

Formal Approach to Analysis

- **For Loops: Formally**
 - In general, a for loop translates to a summation. The index and bounds of the summation are the same as the index and bounds of the for loop.



```
for (i = 1; i <= N; i++) {  
    sum += a[i];  
}
```

sum

Σ

$$\sum_{i=1}^N a[i]$$

- **Nested Loops: Formally**

- Nested for loops translate into multiple summations, one for each for loop.

```
for (int i = 1; i <= N; i++) {  
    for (int j = 1; j <= M; j++) {  
        sum = sum + i + j;  
    }  
}
```

$$\sum_{i=1}^N \sum_{j=1}^M 2 = \sum_{i=1}^N 2M = 2MN$$

- **Consecutive Statements: Formally**
- Add the running times of the separate blocks of your code

```
for (int i = 1; i <= N; i++) {  
    sum = sum+i;  
}  
for (int i = 1; i <= N; i++) {  
    for (int j = 1; j <= N; j++) {  
        sum = sum+i+j;  
    }  
}
```

$$\left[\sum_{i=1}^N 1 \right] + \left[\sum_{i=1}^N \sum_{j=1}^N 2 \right] = N + 2N^2$$

Conditionals: Formally

If (test) s1

else s2:

- Compute the maximum of the running time for s1 and s2.

```

if(test==1) {
    for(int i=1; i<=N; i++) {
        sum=sum+i;
    }
}
else for(int i=1; i<=N; i++) {
    for(int j=1; j<=N; j++) {
        sum=sum+i+j;
    }
}

```

$$\max\left(\sum_{i=1}^N 1, \sum_{i=1}^N \sum_{j=1}^N 2\right) =$$

$$\max(N, 2N^2) = 2N^2$$


```

if(test==1) {
    for(int i=1; i<=N; i++) {
        sum=sum+i;
    }
}
else for(int i=1; i<=N; i++) {
    for(int j=1; j<=N; j++) {
        sum=sum+i+j;
    }
}

```

$$\max \left(\sum_{i=1}^N 1, \sum_{i=1}^N \sum_{j=1}^N 2 \right) =$$

$$\max(N, 2N^2) = 2N^2$$

Algorithm Analysis Categories

- An algorithm may run faster on certain data sets than others
- Algorithms must be examined under different situations to correctly determine their efficiency for accurate comparison
- Three categories to analyze algorithms
 - Best case
 - Worst case
 - Average case

Best Case Analysis

- Best case is defined as which input of size n is the cheapest among all inputs of size n
- Best case is obtained from the in the input data set that results in best possible performance

Best Case Analysis

- **Input**

- Assumes the input data are found in the most advantageous order for the algorithm
- **For sorting**, the most advantageous order is that data are arranged in the required order
- **For searching**, the best case is that the required item is found at the first position
- Question
 - Which order is the most advantageous for an algorithm that sorts 2 5 8 1 4 (ascending order)

- **Input size**

- Assumes that the input size is the minimum possible

Best Case Analysis

- This case executes or causes the fewest number of executions
- So, it computes the lower bound of $T(n)$ and You can not do better
- Is the amount of time the algorithm takes on the smallest possible set of inputs
- Best case behavior is usually not informative and hence usually uninteresting (why?)
- The function is denoted by $T_{\text{best}}(n)$ – best case running time of an algorithm for n inputs

Worst Case Analysis

- Is the complexity of an algorithm based on worst input of each size
- Is the amount of time the algorithm takes on the worst possible set of inputs
- Is the longest running time for any input of size n

Worst Case Analysis

- **Input (i.e., arrangement of data)**
 - Assumes that the data are arranged in the most **disadvantageous** order
 - **For sorting**, the worst case assumes that data are arranged in opposite order
 - **For searching**, the worst case assumes that the required item is found at last position or missing
- **Input size**
 - Assumes the input size to be infinite
 - That is, it assumes that the input size is a very large number
 - That is, it considers $n \rightarrow \text{infinity}$

Worst Case Analysis

- This case, causes highest number of executions to be performed by the algorithm
- So, it computes the upper bound of $T(n)$, the longest running time for any input size n
- Commonly used for computing $T(n)$
- Is a function and is denoted by $T_{\text{worst}}(n)$ – the worst case running time of the algorithm for n inputs

Worst Case Analysis

- Worst case is easier to analyze and can yield useful information
- In certain application domains (e.g., air traffic control, surgery) knowing of the worst case time complexity is of crucial importance

Average Case Analysis

- Is the complexity of an algorithm averaged over all inputs of each size
- Varies between the best case and worst case
- Gives the average performance of an algorithm
- Computes the **optimal** (the one which is found most of the time) **bound of $T(n)$**
- The average case can be as bad as the worst case

Average Case Analysis

- **Input (i.e., arrangement of data)**
 - Assumes that data are arranged in random order
 - **For searching**, it assumes that the required item is found at any position or can be missing
 - **For sorting**, it assumes that data are arranged in a random order
- **Input size**
 - The input size can be any random number as small as the best case or as large as the worst case

Average Case Analysis

- Problems with performing an average case analysis
 - It may not be apparent what constitutes an “average” input for a particular problem
 - There is no clue as to what can be an average input
 - It is difficult to envision the data that would cause an algorithm (e.g., alg. for insertion sort) to exhibit its average behavior
 - Average case is difficult to determine

Average Case Analysis

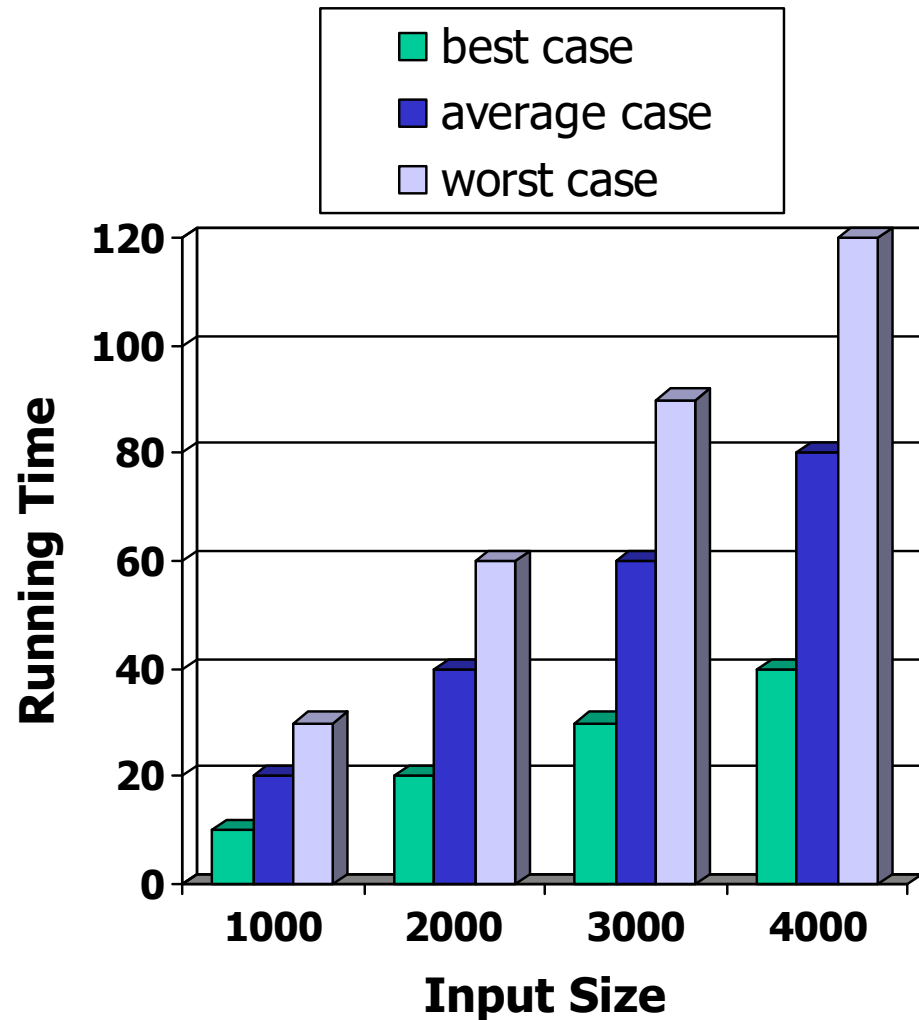
- Problems with performing an average case analysis
 - The average operation count is often
 - Quite difficult to determine or define
 - Quite difficult to analyze
 - Often we assume that all inputs of a given size are equally likely
 - In practice, this assumption may be violated, but randomized algorithms can sometimes force this to hold

Average Case Analysis

- As a result, in several of the algorithms, we limit our analysis to determining the best and worst counts
- We focus on the worst case running time
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics

Graphical view of best, average and worst cases

- The running time of an algorithm varies with the input and typically grows with the input size



Which Analysis to Use?

- Question
 - How can we determine the complexity of an algorithm on any data set?
 - When is the worst case time important?
- Any of these categories have their own pros and cons depending on the data set (arrangement of the data and input size)
 - While average time appears to be the fairest measure, it may be difficult to determine
 - Typically, the worst case analysis is used to estimate the complexity of an algorithm (That is, we compare algorithms at their worst case)

Why use a worst case analysis

- The following are reasons for using worst case analysis as an estimate for the complexity of an algorithm
1. The worst case running time of an algorithm provides an **upper bound on the running time for all inputs (i.e., all cases)**
 - Thus, there is no bad operation than worst case
 - So, knowing it gives us guarantee that the algorithm will never take any longer

Why use a worst case analysis

2. For some algorithms, the worst case occurs fairly often
 - That is , Worst case "may" be a typical case
3. The average case is often roughly as bad as the worst case
 - Average case may be approximately the worst case ($t_j = j / 2$) is roughly quadratic)
4. Often it is hard to pick average cases and compute expected times.
 - The average case bounds are usually more difficult to compute

Conclusion

- Unless otherwise specified, running time is the worst case

Example

- Compute the best, average and worst case time complexities of the Insertion sort

Asymptotic Analysis

- Asymptotic analysis is concerned with how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound.
- There are five notations used to describe a running time function. These are:
 - Big-Oh Notation (O)
 - Big-Omega Notation (Ω)
 - Theta Notation (Θ)
 - Little-o Notation (o)
 - Little-Omega Notation (ω)

The Big-Oh Notation

- It's only concerned with what happens for very a large value of n .
- Therefore only the largest term in the expression (function) is needed.
- Big-Oh is mainly concerned with large values of n
- Big-O expresses an *upper bound* on the growth rate of a function, for sufficiently large values of n .
- **Formal Definition:** $f(n) = O(g(n))$ if there exist $c, k \in \mathcal{R}^+$ such that for all $n \geq k$, $f(n) \leq c \cdot g(n)$.
- **Examples:** The following points are facts that you can use for Big-Oh problems:

$$1 \leq n \quad \text{for all } n \geq 1$$

$$n \leq n^2 \quad \text{for all } n \geq 1$$

$$2^n \leq n! \quad \text{for all } n \geq 4$$

$$\log_2 n \leq n \quad \text{for all } n \geq 2$$

$$n \leq n \log_2 n \quad \text{for all } n \geq 2$$

1. $f(n)=10n+5$ and $g(n)=n$. Show that $f(n)$ is $O(g(n))$.

- To show that $f(n)$ is $O(g(n))$ we must show that constants c and k such that

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq k$$

$$\text{Or } 10n+5 \leq c \cdot n \text{ for all } n \geq k$$

Try $c=15$. Then we need to show that $10n+5 \leq 15n$

Solving for n we get: $5 \leq 5n$ or $1 \leq n$.

So $f(n) = 10n+5 \leq 15 \cdot g(n)$ for all $n \geq 1$.

$(c=15, k=1)$.

2. $f(n) = 3n^2 + 4n + 1$. Show that $f(n) = O(n^2)$.

$4n \leq 4n^2$ for all $n \geq 1$ and $1 \leq n^2$ for all $n \geq 1$

$3n^2 + 4n + 1 \leq 3n^2 + 4n^2 + n^2$ for all $n \geq 1$

$\leq 8n^2$ for all $n \geq 1$

So we have shown that $f(n) \leq 8n^2$ for all $n \geq 1$

Therefore, $f(n)$ is $O(n^2)$ ($c=8, k=1$)

- **Typical Orders**

N	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$
1	1	1	1	1	1	1
2	1	1	2	2	4	8
4	1	2	4	8	16	64
8	1	3	8	24	64	512
16	1	4	16	64	256	4,096
1024	1	10	1,024	10,240	1,048,576	1,073,741,824

Exercise:

$$f(n) = (3/2)n^2 + (5/2)n - 3$$

Show that $f(n) = O(n^2)$

In simple words, $f(n) = O(g(n))$ means that the growth rate of $f(n)$ is less than or equal to $g(n)$.

Big-O Theorems

For all the following theorems, assume that $f(n)$ is a function of n and that k is an arbitrary constant.

Theorem 1: k is $O(1)$

Theorem 2: A polynomial is $O(\text{the term containing the highest power of } n)$.

Polynomial's growth rate is determined by the leading term

If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$

- In general, $f(n)$ is big-O of the dominant term of $f(n)$.

Theorem 3: $k \cdot f(n)$ is $O(f(n))$

Constant factors may be ignored

E.g. $f(n) = 7n^4 + 3n^2 + 5n + 1000$ is $O(n^4)$

- **Theorem 4(Transitivity):** If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$, then $f(n)$ is $O(h(n))$.
- **Theorem 5:** For any base b , $\log_b(n)$ is $O(\log n)$.
- All logarithms grow at the same rate
- $\log_b n$ is $O(\log_d n)$ $b, d > 1$

- **Theorem 6:** Each of the following functions is big-O of its successors:
- k
- $\log_b n$
- n
- $n \log_b n$
- n^2
- n to higher powers
- 2^n
- 3^n
- larger constants to the n th power
- $n!$
- n^n
- $f(n) = 3n \log_b n + 4 \log_b n + 2$ is $O(n \log_b n)$ and $O(n^2)$ and $O(2^n)$

Properties of the O Notation

- Higher powers grow faster

$$n^r = O(n^s) \text{ if } 0 \leq r \leq s$$

- Fastest growing term dominates a sum
- If $f(n)$ is $O(g(n))$, then $f(n) + g(n)$ is $O(g)$

E.g $5n^4 + 6n^3$ is $O(n^4)$

- Exponential functions grow faster than powers,
i.e. $n^k = O(b^n)$, $\forall b > 1$ and $k \geq 0$

E.g. n^{20} is $O(1.05^n)$

- Logarithms grow more slowly than powers
- $\log_b n = O(n^k) \forall b > 1$ and $k \geq 0$

E.g. $\log_2 n$ is $O(n^{0.5})$

Big-Omega Notation

- Just as O-notation provides an asymptotic upper bound on a function, Ω notation provides an asymptotic lower bound.
- **Formal Definition:** A function $f(n)$ is $\Omega(g(n))$ if there exist constants c and $k \in \mathcal{R}^+$ such that

$f(n) \geq c \cdot g(n)$ for all $n \geq k$.

$f(n) = \Omega(g(n))$ means that $f(n)$ is greater than or equal to some constant multiple of $g(n)$ for all values of n greater than or equal to some k .

Example: If $f(n) = n^2$, then $f(n) = \Omega(n)$

- In simple terms, $f(n) = \Omega(g(n))$ means that the growth rate of $f(n)$ is greater than or equal to $g(n)$.

Theta Notation

- A function $f(n)$ belongs to the set of $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be sandwiched between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$, for sufficiently large values of n .
- **Formal Definition:** A function $f(n)$ is $\Theta(g(n))$ if it is both $O(g(n))$ and $\Omega(g(n))$.
- In other words, there exist constants c_1 , c_2 , and $k > 0$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq k$.
- If $f(n) = \Theta(g(n))$, then $g(n)$ is an asymptotically tight bound for $f(n)$.
- In simple terms, $f(n) = \Theta(g(n))$ means that $f(n)$ and $g(n)$ have the same rate of growth.

Example:

1. If $f(n)=2n+1$, then $f(n) = \Theta(n)$

2. $f(n) = 2n^2$ then

$$f(n)=O(n^4)$$

$$f(n)=O(n^3)$$

$$f(n)= O(n^2)$$

- All these are technically correct, but the last expression is the best and tight one. Since $2n^2$ and n^2 have the same growth rate, it can be written as $f(n)= \Theta(n^2)$.

- **Little-o Notation**
- Big-Oh notation may or may not be asymptotically tight, for example:
- $2n^2 = O(n^2)$
- $=O(n^3)$
- $f(n)=o(g(n))$ means for all $c>0$ there exists some $k>0$ such that $f(n)<c.g(n)$ for all $n\geq k$. Informally, $f(n)=o(g(n))$ means $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity.
- **Example:** $f(n)=3n+4$ is $o(n^2)$
- In simple terms, $f(n)$ has less growth rate compared to $g(n)$.
- $g(n)= 2n^2$ $g(n) =o(n^3)$, $O(n^2)$, $g(n)$ is not $o(n^2)$.

- **Little-Omega (ω notation)**
- Little-omega (ω) notation is to big-omega (Ω) notation as little-o notation is to Big-Oh notation. We use ω notation to denote a lower bound that is not asymptotically tight.
- **Formal Definition:** $f(n) = \omega(g(n))$ if there exists a constant $c > 0$ such that $0 < c \cdot g(n) < f(n)$ for all $n \geq k$.

Example: $2n^2 = \omega(n)$ but it's not $\Omega(n)$.

Relational Properties of the Asymptotic Notations

- **Transitivity**

- if $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ then $f(n) = \Theta(h(n))$,
- if $f(n) = O(g(n))$ and $g(n) = O(h(n))$ then $f(n) = O(h(n))$,
- if $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ then $f(n) = \Omega(h(n))$,
- if $f(n) = o(g(n))$ and $g(n) = o(h(n))$ then $f(n) = o(h(n))$, and
- if $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ then $f(n) = \omega(h(n))$.

Symmetry

$f(n)=\Theta(g(n))$ if and only if $g(n)=\Theta(f(n))$.

Transpose symmetry

$f(n)=O(g(n))$ if and only if $g(n)=\Omega(f(n))$,

$f(n)=o(g(n))$ if and only if $g(n)=\omega(f(n))$.

Reflexivity

$f(n)=\Theta(f(n))$,

$f(n)=O(f(n))$,

$f(n)=\Omega(f(n))$.